

#ASLI ENGINEERING

Ziplist and Quicklist

SWIPE

BY

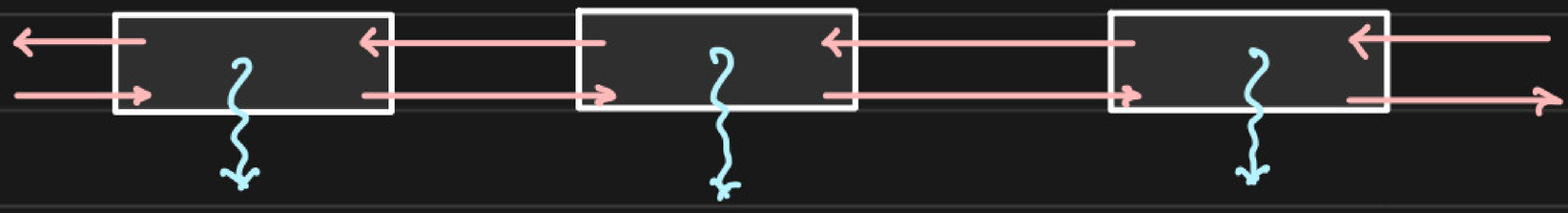
ARPIT BHAYANI

List Internals: Ziplist and Quicklist

Regular list implementation → Doubly linked list

- insert remove from end & middle in O(1)
- traverse back and forth

But this is inefficient because of mem overhead!



Each node contains 3 pointers → next, prev, value

= 4x3 = 12B (on 32 bit machine)

= 8x3 = 24B (on 64 bit machine)

In Redis, each value is a RedisObject that contains

32 bit metadata, ref count, pointer to value ~ 16B

All in all, for every value in a Doubly linked list,

Redis has a 40B overhead!!

a 5 char string in a list would be



10B data + 40B metadata

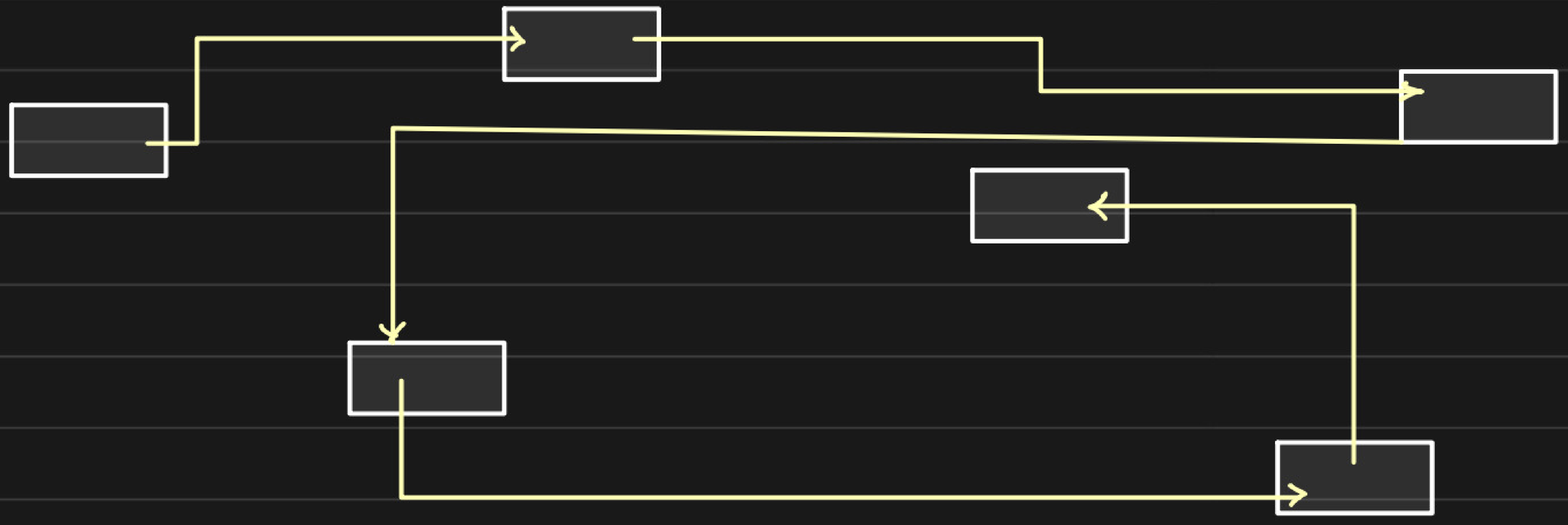
↓ String repr ↓ Node

ARPIT BHAYANI

Disadvantage:

Huge memory overhead for smaller value
Chained pointers are not cache efficient
↳ Nodes are not contiguous allocated
↳ a lot of random reads

How DLL is actually allocated ↓



Ziplist

Ziplist is a solid block of memory where all elements of the list are smooshed together. smaller values fewer elements

Smaller list, hash and sorted sets are implemented using ziplist. Once the threshold is breached they can move to common implementation.

```
CONFIG GET *ziplist*
```

Redis 7.x has a bunch of changes

ARPIT BHAYANI